Using Parallelism to Determine New state Boundaries

Brook Hamilton

hamilbro@oregonstate.edu

CS 475: Parallel Programming

Project 3

Professor Jesse Rosenzweig

October 18, 2026

Oregon State University - Cascades

Introduction:

This project demonstrates real world applications of parallelism by showing the use of parallel data computation in order to give real applicable information to solve a problem. In this case we are trying to determine where new state boundaries should be based on the number of cities in a relative region, amounting to NUMCAPITALS new states. As well as a bonus exercise that instead re-uses existing cities to be designated as capitals instead of creating brand new ones. This is both a better application of the data collected, as well as proof that the prior calculations worked as intended.

The calculations and groupings in this program are accomplished by using the k-means algorithm. The K-means algorithm works by “randomly” picking NUMCAPITALS to be the initial state capitals and records their longitudes, and latitudes as the list of capitals. It then goes through the cities to figure out which NUMCAPITALS they are closest to by using Pythagorean distance. All of the cities in each group's longitudes and latitudes are then averaged which becomes the new set of capitals, this is then repeated until each cities state membership no longer changes, which in our case will be 100 times to be safe.

The data being calculated come from the US Census Bureau which collects data on the 333 largest cities in the US (we are using 331 - no Alaska or Hawaii as it will skew the data).

Machine Details:

Machine: Lenovo LOQ AMD Ryzen 7 7435HS
Architecture: X86_64
CPU(s): 16
Thread(s) per core: 2
Core(s) per socket: 8
Operating System: Windows 11
Compiler: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

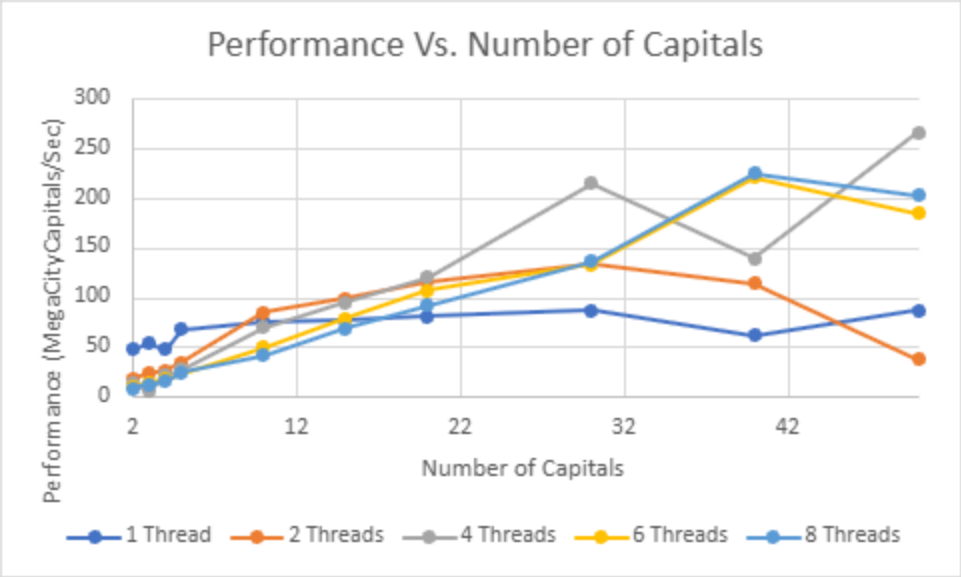
Results:

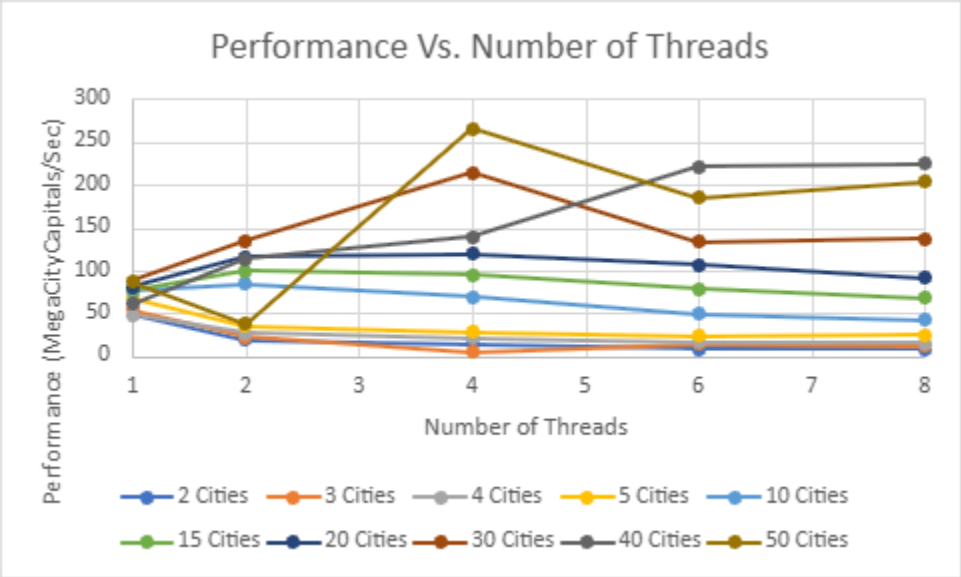
Performance Data

Threads	Cities	Capitals	MegaCityCapitalsPerSec
1	331	2	47.884
1	331	3	53.871
1	331	4	47.59

1	331	5	67.364
1	331	10	75.791
1	331	15	77.171
1	331	20	80.608
1	331	30	87.192
1	331	40	61.435
1	331	50	86.761
2	331	2	18.327
2	331	3	22.773
2	331	4	26.744
2	331	5	34.595
2	331	10	84.685
2	331	15	99.246
2	331	20	115.913
2	331	30	134.582
2	331	40	113.62
2	331	50	37.07
4	331	2	14.459
4	331	3	4.777
4	331	4	21.639
4	331	5	27.602
4	331	10	69.462
4	331	15	94.871
4	331	20	119.546
4	331	30	214.513

4	331	40	139.03
4	331	50	265.765
6	331	2	8.752
6	331	3	13.899
6	331	4	16.855
6	331	5	23.219
6	331	10	49.502
6	331	15	78.64
6	331	20	106.904
6	331	30	133.037
6	331	40	221.268
6	331	50	184.389
8	331	2	7.878
8	331	3	12.032
8	331	4	15.048
8	331	5	24.474
8	331	10	41.895
8	331	15	68.437
8	331	20	91.783
8	331	30	136.414
8	331	40	224.822
8	331	50	203.005


Performance Vs. Number of Capitals
Performance (MegaCityCapitals/Sec)
300
250
200
150
100
50
0
2
12
22
32
42
Number of Capitals
1 Thread
2 Threads
4 Threads
6 Threads
8 Threads


Performance Vs. Number of Threads
Performance (MegaCityCapitals/Sec)
300
250
200
150
100
50
0
1
2
3
4
5
6
7
8
Number of Threads
2 Cities
3 Cities
4 Cities
5 Cities
10 Cities
15 Cities
20 Cities
30 Cities
40 Cities
50 Cities

Extra Credit:

```
 0:   74.08 ,   40.68 , JerseyCity,NJ
 1:   98.10 ,   26.77 , Edinburg,TX
 2:   83.90 ,   39.64 , Dayton,OH
 3:   85.24 ,   33.60 , SouthFulton,GA
 4:   85.45 ,   36.84 , Louisville,KY
 5:  119.60 ,   36.65 , Clovis,CA
 6:   77.31 ,   36.38 , NewportNews,VA
 7:  119.04 ,   35.35 , Bakersfield,CA
 8:   83.74 ,   42.42 , AnnArbor,MI
 9:   82.42 ,   28.83 , Gainesville,FL
10:   80.43 ,   34.46 , Columbia,SC
11:  114.74 ,   36.40 , NorthLasVegas,NV
12:   85.94 ,   40.50 , Fishers,IN
13:  112.00 ,   33.37 , Tempe,AZ
14:  122.26 ,   47.55 , Renton,WA
15:  117.33 ,   34.47 , Victorville,CA
16:  117.44 ,   34.03 , JurupaValley,CA
17:  108.55 ,   45.79 , Billings,MT
18:  101.58 ,   33.02 , Lubbock,TX
19:   88.97 ,   41.99 , Rockford,IL
20:   81.06 ,   41.00 , Akron,OH
21:   91.08 ,   31.55 , BatonRouge,LA
22:   88.77 ,   37.08 , Clarksville,TN
23:  104.93 ,   39.71 , Denver,CO
24:  122.17 ,   37.92 , Berkeley,CA
25:   71.21 ,   42.29 , Cambridge,MA
26:  117.13 ,   32.95 , SanDiego,CA
27:   95.18 ,   29.78 , Pasadena,TX
28:   77.54 ,   43.03 , Rochester,NY
29:   76.90 ,   39.01 , Washington,DC
30:   94.34 ,   44.32 , Minneapolis,MN
31:   75.31 ,   40.30 , Philadelphia,PA
32:  116.40 ,   43.60 , Meridian,ID
33:  121.67 ,   37.41 , SanJose,CA
34:   97.61 ,   30.46 , RoundRock,TX
35:   95.04 ,   38.65 , Olathe,KS
36:  120.93 ,   39.10 , Roseville,CA
37:  111.98 ,   40.69 , WestValleyCity,UT
```

38: 73.03 , 41.77 , Waterbury,CT
39: 111.65 , 40.25 , Provo,UT
40: 117.17 , 33.58 , Murrieta,CA
41: 117.33 , 47.67 , Spokane,WA
42: 119.63 , 34.47 , Ventura,CA
43: 106.64 , 33.64 , LasCruces,NM
44: 80.40 , 26.36 , CoralSprings,FL
45: 117.93 , 33.85 , Fullerton,CA
46: 118.38 , 34.23 , Burbank,CA
47: 96.94 , 33.22 , Frisco,TX
48: 122.58 , 45.04 , Salem,OR
49: 122.87 , 38.09 , SantaRosa,CA

Conclusion:

The results of the calculations and groupings shown by the performance graphs are very interesting. As shown, the peak performance was achieved by using 4 threads with outstanding performance happening at 50 capitals worth of data. Although 4 threads also had one of the worst performances happening at 2 capitals along with many other threads counts, this is most likely because of the large amount of overhead to work ratio. There is also a notable dip in performance when 4 threads run the calculations on 40 capitals, which is particularly interesting because it then does better again at 50 capitals worth of data. This is of note because although 4 threads out performs 6 and 8 threads for most of the work, it falls behind at 40 while 6 and 8 threads peak here and then start to plateau at 50 capitals. while 4 threads jumping to 265.8 MegaCityCapitals/Sec 6 and 8 drop.

It is also interesting to see that for small workloads (2–5 capitals), a single thread performs best, and every thread added makes performance worse. At 2 capitals 1 thread reached 47.9 MegaCityCapitals/sec, while 8 threads dropped to 7.9. This is most likely because every city enters the `#pragma omp critical` section once per iteration. With only a few capitals, each city does just a few distance calculations before waiting for the lock, so the threads spend more and more time waiting on each other. As the number of capitals increases, each city does more work between critical sections, and adding threads starts to pay off:

- 2 to 4 threads perform best at 10–20 capitals
- 4 to 8 threads perform best at 30 capitals and above

The data supplied resulted in odd results especially reflected in the extra credit work. Despite the capitals being distributed across the United States 13/50 of the new capitals are located in

California. This is because the data is made up of the 331 most populated cities, many of which reside in California. Which shows that the data was processed through the K-means algorithm correctly and also that if this were a real application, the data would probably need to be collected and organized differently to avoid such a lop-sided country.

In conclusion this real world application of parallelism was very interesting to experiment with, and it was interesting to investigate further into what causes slow downs/speed ups when experimenting with large datasets and multiple threads. The addition of an explicit critical section also helps deepen my understanding of how threads work together and what can be an issue as the size of the data fluctuates.